



## FINAL PROJECT

### ECE 484: Embedded Systems Design

|                     | Score                | Signature |
|---------------------|----------------------|-----------|
| Name: Gabe Litteken |                      |           |
| Name: James Nipper  |                      |           |
| Name:               |                      |           |
| Demonstration       | A 75<br>B 10<br>E 25 | <br><br>  |

110

### Overview

For your final project, you will design an embedded system that will serve as an alarm system/smart home hub. The embedded system will encapsulate all the concepts covered throughout the semester.

### Deliverables

1. System Demonstration (100 points)
2. Completed Worksheet submitted to Canvas (40 points)
3. Commented Source Code submitted to Canvas (10 points)

### Instructions

Your embedded system will encapsulate a variety of behaviors that are broken out into various modules. **You are required to complete Module A** and you can choose from the remaining modules to earn up to 100 points for your demonstration. **Due to pin constraints, you will not be able to complete all four modules.** You can pick wither Module B or Module C for your 10 point module. You will also pick Module D or Module E which are worth 15 points and 25 points, respectively. The maximum possible score for this project is 110/100 (Module A, B or C, and E). A description and design specifications for each module are given below:

#### Module A [75 points] – Alarm System

This module will contain the core functionality of your system. Your alarm system will either be in the "ARMED" or "DISARMED" state which can be changed by entering a code on three pushbuttons. You will choose a three digit code to arm the system and a different three digit code to disarm the system. Your system will start in the "DISARMED" state and can be armed by inputting your three button code. You will display the input code on the second line of the LCD. Any incorrect sequence should clear the code before a new code can be input. Once the correct code is input, you will implement a 10 second countdown using time multiplexing. The 10 second countdown should be displayed in the last two digits of the second row of the LCD (connected to TWI/I2C). After 10 seconds, your system will initiate a "lock door" process which involves rotating a stepper motor 90° CW. During this "lock" process, your system will not receive any additional code inputs. Once the door is "locked", you will light a red LED; turn off a green LED; and your system will be considered to be in the "ARMED" state. You will display



"ARMED" on the first line of the LCD when in the "ARMED" state.

Once your system is in the "ARMED" state, you can disarm the system by inputting a different three button code. Once again, you will display the code on the LCD and clear the screen whenever an incorrect code has been entered. However, an incorrect code will also trigger the alarm. When the alarm is triggered, you will need to use time multiplexing to blink the red LED every 250ms; sound an active buzzer; and display "ALARM" on the first line of your LCD. If the correct code is input, your system will start the "unlock" process that will see the stepper motor rotated 90° CCW. After the "unlock" process has completed, you will turn off the red LED and light the green LED and your LCD should display "DISARMED" on the first line of the LCD. At this point your system has returned to the "DISARMED" state.

This module will be implemented using Timer 1 and Timer 1 will serve as the system clock for your system. Within the Timer 1 ISR, you should debounce the inputs from the three pushbuttons and write a state machine to implement the aforementioned behavior of this module. For this module, you will need to determine the proper time interval for Timer 1. You will need to provide justification for the chosen time interval in the design questions.

#### **Module B [10 points] – Light Dimmer Switch**

For this module, you will implement a light dimming control behavior using a pushbutton and a potentiometer. The pushbutton will control whether a blue LED is ON or OFF while the potentiometer will control the brightness level of the blue LED. The pushbutton will utilize external interrupt INT0 while the potentiometer will be connected to an analog channel of your choice. You will need to convert the analog voltage from the potentiometer to a quantized digital voltage. To reduce noise, you will use the 8 MSB of the quantized voltage. You will then convert the quantized voltage to its analog equivalent and display the analog voltage in the last four digits of the first row of the LCD screen.

#### **Module C [10 points] – Fan Control**

For this module, you will implement fan control using a pushbutton and the PWM module on Timer 0. The PWM on Timer 0 will be configured to have wave frequency of 500Hz. The duty cycle of the PWM wave will be controlled by the pushbutton that is connected to external interrupt INT0. Every press of the pushbutton will increment the duty cycle of the fan by 10% up to 100% duty cycle. Once the duty cycle reaches 100%, another button press will cause the duty cycle to decrement by 10% until the duty cycle reaches 0%. The cycle will then repeat upon subsequent button presses.

#### **Module D [15 points] – Alarm HQ**

For this module, you will augment the state machine of Module A with additional communication capabilities. When an alarm has been triggered (i.e. the system is in the "ARMED" state and an incorrect code is entered), your system will send a code to the Alarm HQ via SPI. The Alarm HQ will be an embedded system provided by Dr. Swanson. When your alarm is tripped, it should send a code of 1010 1010 to the Alarm HQ. Upon proper receipt of this code, the Alarm HQ will display "ALARM ON" on its LCD screen. When your alarm is subsequently disarmed, it should send a code of 0101 0101 to the Alarm HQ. Upon proper receipt of this code, the Alarm HQ will display "ALARM FF" on its LCD screen.

You will need to configure your system to operate in SPI master mode for this module. After a transmission is complete your system will trigger an interrupt. You will configure your SPI

module to have its clock default to high when the bus is not in use and the bus will be sampled on a falling edge of the clock. The clock for the SPI bus will operate at a frequency of 2MHz.

### Module E [25 points] – Help Button

For this module, you will augment the state machine of Module A with additional communication capabilities. You will include a fourth pushbutton in your system that will serve as a “help” button. This pushbutton will be monitored by external interrupt INT1. When the “help” button is pressed, you will send a code 1111 0000 to the Alarm HQ via USART. The Alarm HQ will be an embedded system provided by Dr. Swanson. Upon proper receipt of this code, the Alarm HQ will display “HELP!” on its LCD screen. Pressing the red push button on the Alarm HQ will send a data packet of 0000 1111 back to your system. Upon receipt of this code, your system will display “OK” on your LCD screen for 2 seconds using a delay. After this delay, you will display the current status of the alarm on your LCD screen and clear code currently being input to your system.

For this system, you will configure your system in USART multiprocessor mode with a baud rate of 250kbps. Your system will use 8-bit data packets with no parity and will be initialized in transmitter mode. Once a transmission is complete, you will need to configure your system in receiver mode. Similarly, once a message is received, you will need to configure your system in transmitter mode. You will utilize Tx complete and Rx complete ISRs to implement this module.



## Demonstration Rubric

Please demonstrate your system to Dr. Swanson. Checkpoint scores are provided in the following rubric. Scores between checkpoints may be awarded if a student has exceeded the previous checkpoint but has not reached the next checkpoint(s).

### MODULE A RUBRIC:

| Score | Reasoning                                                                                                                                          |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 75.0  | Student has a fully operational Module A.                                                                                                          |
| 65.0  | Student can properly transition system from the "ARMED" to "DISARM" state. The system does not properly handle incorrect code inputs.              |
| 55.0  | Student can properly transition system from the "DISARM" to "ARM" state. The system properly handles incorrect code inputs and triggers the alarm. |
| 45.0  | Student can properly transition system from the "DISARM" to "ARM" state. The system does not properly handle incorrect code inputs.                |
| 35.0  | Student can input a code and display it to the LCD. The system does not properly transition from the "DISARM" state to the "ARMED" state.          |
| 25.0  | Student has properly wired all necessary circuitry.                                                                                                |
| 0.0   | Student cannot program their microcontroller and has not installed microchip studio or has functional code in their while(1) loop.                 |

### MODULE B RUBRIC:

| Score | Reasoning                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------|
| 10.0  | Analog voltage is properly displayed on the LCD screen.                                                                            |
| 8.0   | Quantized voltage is properly displayed on the LCD screen.                                                                         |
| 4.0   | Pushbutton turns ON and OFF the blue LED.                                                                                          |
| 2.0   | Student has properly wired all necessary circuitry.                                                                                |
| 0.0   | Student cannot program their microcontroller and has not installed microchip studio or has functional code in their while(1) loop. |

### MODULE C RUBRIC:

| Score | Reasoning                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------|
| 10.0  | Student can decrement duty cycle by 10% down to 0% by pressing button. Current duty cycle is properly displayed on the LCD screen. |
| 8.0   | Student can increment duty cycle by 10% up to 100% by pressing button. Current duty cycle is properly displayed on the LCD screen. |
| 6.0   | Student can adjust duty cycle by pressing button. Current duty cycle is not displayed on the LCD screen.                           |
| 4.0   | Student can spin motor at 100% duty cycle by connecting directly to +5V.                                                           |
| 2.0   | Student has properly wired all necessary circuitry.                                                                                |
| 0.0   | Student cannot program their microcontroller and has not installed microchip studio or has functional code in their while(1) loop. |

**MODULE D RUBRIC:**

| Score | Reasoning                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------|
| 15.0  | Student clears any current input code and displays "ARMED" on the LCD screen.                                                      |
| 12.0  | Student properly receives message from Alarm HQ.                                                                                   |
| 9.0   | Student properly sends message to Alarm HQ.                                                                                        |
| 6.0   | Student has properly configured the SPI module                                                                                     |
| 3.0   | Student has properly wired all necessary circuitry.                                                                                |
| 0.0   | Student cannot program their microcontroller and has not installed microchip studio or has functional code in their while(1) loop. |

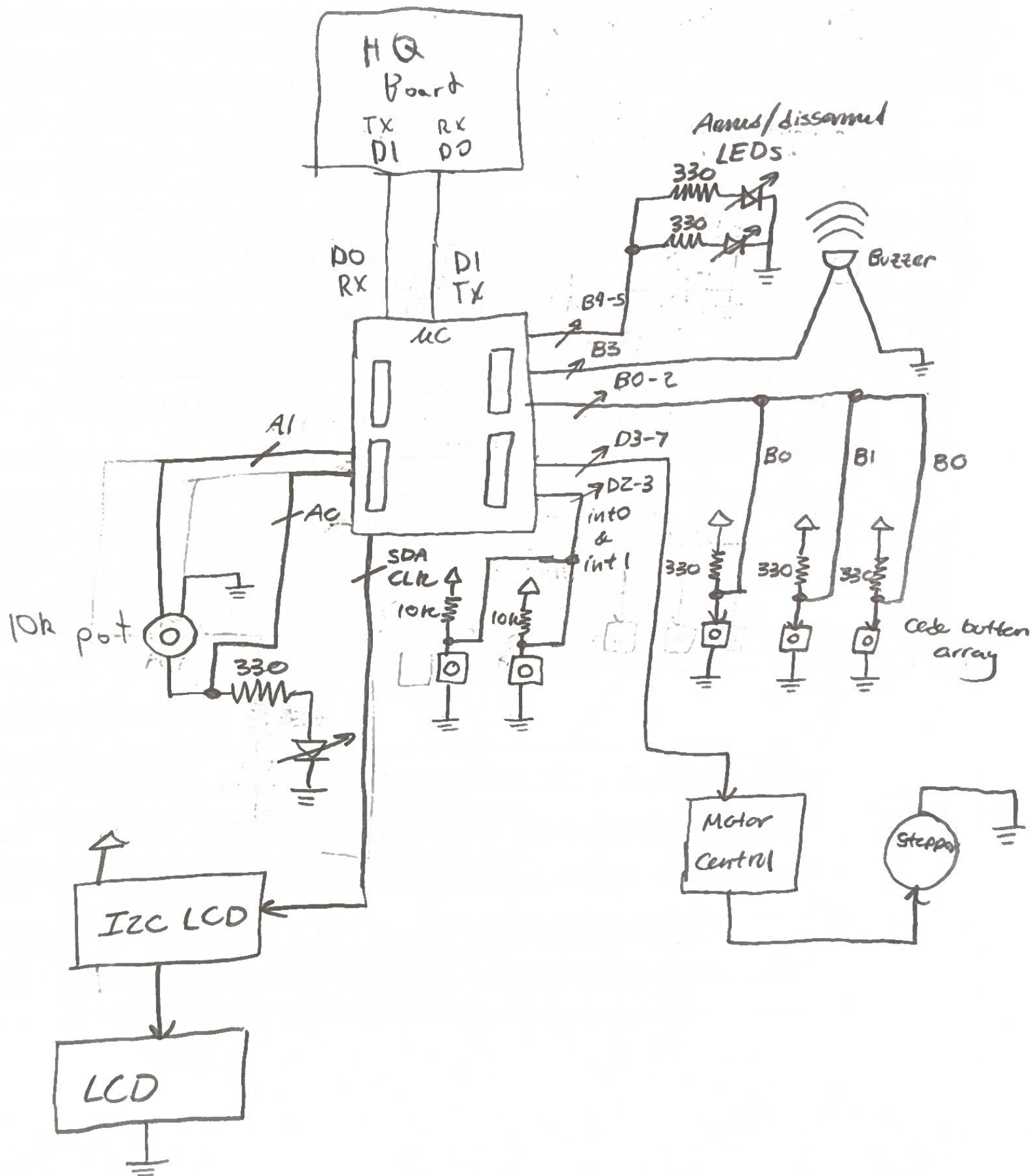
**MODULE E RUBRIC:**

| Score | Reasoning                                                                                                                                   |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 25.0  | Student clears any current input code and displays "OK" on the LCD screen for 2 seconds before the system transitions to the "ARMED" state. |
| 20.0  | Student properly receives message from Alarm HQ.                                                                                            |
| 15.0  | Student properly sends message to Alarm HQ.                                                                                                 |
| 5.0   | Student has properly configured the USART module                                                                                            |
| 3.0   | Student has properly wired all necessary circuitry.                                                                                         |
| 0.0   | Student cannot program their microcontroller and has not installed microchip studio or has functional code in their while(1) loop.          |



## Circuit Diagram

[10 points] Draw a circuit diagram for your system. Make sure to label all major modules, resistance values, and signal lines.



## Design Questions

1. [10 points] Fill out the following table of GPIO pins and label the functionality of each pin in your system.

| Pin   | Functionality                  |
|-------|--------------------------------|
| PIND7 | Motor controller IN4           |
| PIND6 | " " IN3                        |
| PIND5 | " " IN2                        |
| PIND4 | " " IN1                        |
| PIND3 | Emergency button (module E)    |
| PIND2 | Variable LED toggle (module B) |
| PIND1 | USART TX                       |
| PIND0 | USART RX                       |

| Pin   | Functionality                |
|-------|------------------------------|
| PINB5 | Green LED (on when disarmed) |
| PINB4 | Red LED (on when armed)      |
| PINB3 | Buzzer                       |
| PINB2 | Code button C                |
| PINB1 | Code button B                |
| PINB0 | Code button A                |

| Pin   | Functionality      |
|-------|--------------------|
| PINC0 | ADC input pin      |
| PINC1 | variable LED power |
| PINC2 | NC                 |
| PINC3 | NC                 |
| PINC4 | SDA                |
| PINC5 | SCL                |

2. [10 points] At what interval did you choose to trigger an ISR on Timer 1? Explain the design considerations for choosing this interval.

We chose to trigger an ISR every 1 ms. This choice of timing allowed for a relatively long ISR, while also sampling button presses correctly. In addition, a 1 ms ISR allows for easy time multiplexing into seconds.



3. [10 points] Draw a block diagram of the state machine you implemented for Timer 1. Make sure to label the possible transitions for each state and any outputs of a given state.

